==========

DSE501 : Introduction to Data Science Tools
Prepared by : Prof. Dr. Hasan DAĞ
Date Assigned : 03.11.2026
Exam Type : **Quiz (**in place of Homework 1a & 1b, String Manipulation, arrays and struct data type**)**
Name and last name :

==========

Instructions: Answer the following 2 questions in writing using a pen or pencil. You must justify your logic by explaining the underlying memory operations and string pointer mechanics.

Question 1 (20 points): String Parsing & Buffer Mechanics (Homework 1a & 1b)

Consider the following snippet executed while reading a record line like **`"Hasan, Dag, 61, M, Cibali, 1, Istanbul, Turkey."`**

```
char *token = strtok(line, ",");
token = strtok(NULL, ",");
strcpy(lastnames[count], token);
```

Part A: What EXACTLY does strtok modify inside the original line memory buffer when it encounters a delimiter character?

- **A)** `strtok` overwrites the matching delimiter character (the comma `,`) in the original `line` buffer with a null terminator byte (`'\0'`) and returns a pointer to the start of the token.

Part B: If a student forgets to handle leading spaces after commas and executes strcmp(lastnames[count], "Dag"), why will strcmp return a non-zero (false) result? State the exact byte sequence stored in lastnames[count].

- **B)** The text in the file is `", Dag"`. The string after the comma contains a leading space byte (`' '`). Thus, `lastnames[count]` contains `" Dag"` (`[' ', 'D', 'a', 'g', '\0']`), which fails exact comparison against `"Dag"`.

Question 2 (20 points): Memory Layout & Swap Mechanics (Homework 1a vs 1b)

In Homework 1a, records were sorted using parallel arrays. In Homework 1b, records were stored and sorted using an array of composite struct objects.

Part A: Explain why swapping two records in Homework 1b requires only a single temporary variable allocation (Instructor temp = instructors[j];), whereas Homework 1a required multiple separate swap operations using strcpy and temporary integer variables.

- **A)** A `struct` forms a single contiguous block of memory containing all fields. Assigning one struct variable to another copies the whole memory block at once, whereas parallel arrays store fields in disconnected memory regions requiring individual field-by-field swaps.

Part B: Assuming sizeof(Address) is 156 bytes and sizeof(Instructor) is 264 bytes, describe what occurs in stack memory when the expression instructors[j] = instructors[j + 1]; is executed. (Discuss byte copying vs pointer swapping).

- **B)** The compiler executes a direct memory block copy (`memcpy` under the hood) of all 264 contiguous bytes from `instructors[j+1]` into `instructors[j]`.

Instructions: *Write complete C function implementation code directly on the blank lines provided below using a pen or pencil. Pay close attention to function return types, string parsing logic, pointer mechanics, and exact struct array indexing.*

Question 3 (30 Points): Parsing a Line into Parallel Arrays (Void Function)

Write a void C function named parseLineToArrays that receives a single line string containing formatted instructor data (e.g., "Hasan, Dag, 61, M, Cibali, 1, Istanbul, Turkey."), an index integer idx, and three parallel arrays: names, lastnames, and ages.

The function must break the line, trim leading spaces, store the name and lastname into their respective 2D character arrays at position idx, and convert and store the age into the integer array at position idx.

```
void parseLineToArrays(char *line, int idx,
                       char names[][50], char lastnames[][50], int ages[]);
```

Your Code Solution (Q3):

```
void parseLineToArrays(char *line, int idx,
                       char names[][50], char lastnames[][50], int ages[]) {
    // 1. Tokenize Name
    char *token = strtok(line, ",");
    if (token) {
        strcpy(names[idx], token);
    }

    // 2. Tokenize Last Name & trim leading space
    token = strtok(NULL, ",");
    if (token) {
        while (*token == ' ') token++; // Move pointer past leading space
        strcpy(lastnames[idx], token);
    }

    // 3. Tokenize Age & convert to int
    token = strtok(NULL, ",");
    if (token) {
        ages[idx] = atoi(token);
    }
}
```

Question 4 (30 Points): Parsing a Line into a Struct Array (Void Function)

Repeat the parsing requirement from Question 1, but this time using an array of Instructor structures.

Given the struct definition below, write a void C function named parseLineToStructArray that receives a single formatted data string line, an index integer idx, and an array of Instructor structs. The function must parse and populate the name, lastname, and age struct members at position idx.

```
typedef struct {
    char name[50];
    char lastname[50];
    int age;
} Instructor;
void parseLineToStructArray(char *line, int idx, Instructor instructors[]);
```

Your Code Solution (Q4):

```
void parseLineToArrays(char *line, int idx,
                       char names[][50], char lastnames[][50], int ages[]) {
    // 1. Tokenize Name
    char *token = strtok(line, ",");
    if (token) {
        strcpy(names[idx], token);
    }

    // 2. Tokenize Last Name & trim leading space
    token = strtok(NULL, ",");
    if (token) {
        while (*token == ' ') token++; // Move pointer past leading space
        strcpy(lastnames[idx], token);
    }

    // 3. Tokenize Age & convert to int
    token = strtok(NULL, ",");
    if (token) {
        ages[idx] = atoi(token);
    }
}
```